

[Guides](#) > [Data engineering](#) > [Data loading](#) > [Overview](#)

# Snowpipe

Snowpipe enables loading data from files as soon as they're available in a stage. This means you can load data from files in micro-batches, making it available to users within minutes, rather than manually executing COPY statements on a schedule to load larger batches.

## How does Snowpipe work?

Snowpipe loads data from files as soon as they are available in a stage. The data is loaded according to the COPY statement defined in a referenced pipe.

A pipe is a named, first-class Snowflake object that contains a COPY statement used by Snowpipe. The COPY statement identifies the source location of the data files (i.e., a stage) and a target table. All data types are supported, including semi-structured data types such as JSON and Avro.

Different mechanisms for detecting the staged files are available:

- Automating Snowpipe using cloud messaging

Automated data loads leverage event notifications for cloud storage to inform Snowpipe of the arrival of new data files to load. Snowpipe polls the event notifications from a queue. By using the metadata in the queue, Snowpipe loads the new data files into the target table in a continuous, serverless fashion based on the parameters defined in a specified pipe object.

- Calling Snowpipe REST endpoints

Your client application calls a public REST endpoint with the name of a pipe object and a list of data filenames. If new data files matching the list are discovered in the stage referenced by the pipe object, they are queued for loading. Snowflake-provided compute resources load data from the queue into a Snowflake table based on parameters defined in the pipe.

## Supported Cloud Storage services

The following table indicates the cloud storage service support for automated Snowpipe and Snowpipe REST API calls from Snowflake accounts hosted on each [cloud platform](#):

| Snowflake Account Host | Amazon S3 | Google Cloud Storage | Microsoft Azure Blob storage | Microsoft Data Lake Storage Gen2 | Microsoft Azure General-purpose v2 |
|------------------------|-----------|----------------------|------------------------------|----------------------------------|------------------------------------|
| Amazon Web Services    | ✓         | ✓                    | ✓                            | ✓                                | ✓                                  |
| Google Cloud           | ✓         | ✓                    | ✓                            | ✓                                | ✓                                  |
| Microsoft Azure        | ✓         | ✓                    | ✓                            | ✓                                | ✓                                  |

For more information, see [Automate continuous data loading with cloud messaging](#) and [Overview of the Snowpipe REST endpoints to load data](#).

Note that the government regions of the cloud providers do not allow event notifications to be sent to or from other commercial regions. For more information, see [AWS GovCloud \(US\)](#) and [Azure Government](#).

### Important

Snowflake recommends that you enable cloud event filtering for Snowpipe to reduce costs, event noise, and latency. For more information about configuring event filtering for each cloud provider, see the following pages:

- [Configuring event notifications using object key name filtering - Amazon S3](#)
- [Understand event filtering for Event Grid subscriptions - Azure](#)
- [Filtering messages - Google Pub/Sub](#)

## How is Snowpipe different from bulk data loading?

This section briefly describes the primary differences between Snowpipe and a bulk data load workflow using the COPY command. Additional details are provided throughout the Snowpipe documentation.

### Authentication

**Bulk data load** Relies on the security options supported by the client for authenticating and initiating a user session.

**Snowpipe** ***When calling the REST endpoints:*** Requires key pair authentication with JSON Web Token (JWT). JWTs are signed using a public/private key pair with RSA encryption.

## Load history

**Bulk data load** Stored in the metadata of the target table for 64 days. Available upon completion of the COPY statement as the statement output.

**Snowpipe** Stored in the metadata of the pipe for 14 days. Must be requested from Snowflake via a REST endpoint, SQL table function, or ACCOUNT\_USAGE view.

### Important

To avoid reloading files (and duplicating data), we recommend loading data from a specific set of files using ***either*** bulk data loading or Snowpipe but not both.

## Transactions

**Bulk data load** Loads are always performed in a single transaction. Data is inserted into table alongside any other SQL statements submitted manually by users.

**Snowpipe** Loads are combined or split into a single or multiple transactions based on the number and size of the rows in each data file. Rows of partially loaded files (based on the ON\_ERROR copy option setting) can also be combined or split into one or more transactions.

## Compute resources

**Bulk data load** Requires a user-specified warehouse to execute COPY statements.

**Snowpipe** Uses Snowflake-supplied compute resources.

## Cost

**Bulk data load** Billed for the amount of time each virtual warehouse is active.

**Snowpipe**

Billed according to the compute resources used in the Snowpipe warehouse while loading the files.

## Recommended load file size

For the most efficient and cost-effective load experience with Snowpipe, we recommend following the file sizing recommendations in [File sizing best practices](#) and staging files once per minute. This approach typically leads to a good balance between cost (i.e. resources spent on Snowpipe queue management and the actual load) and performance (i.e. load latency). For more information, see [Continuous data loads — that is, Snowpipe — and file sizing](#).

## Load order of data files

For each pipe object, Snowflake establishes a single queue to sequence data files awaiting loading. As new data files are discovered in a stage, Snowpipe appends them to the queue. However, multiple processes pull files from the queue; and so, while Snowpipe generally loads older files first, there is no guarantee that files are loaded in the same order they are staged.

## Data duplication

Snowpipe uses file loading metadata associated with each **pipe object** to prevent reloading the same files (and duplicating data) in a table. This metadata stores the path (i.e. prefix) and name of each loaded file, and prevents loading files with the same name even if they were later modified (i.e. have a different eTag).

## Estimating Snowpipe latency

Given the number of factors that can differentiate Snowpipe loads, it is very difficult for Snowflake to estimate latency. File formats and sizes, and the complexity of COPY statements (including SELECT statement used for transformations), all impact the amount of time required for a Snowpipe load.

We suggest that you experiment by performing a typical set of loads to estimate average latency.

## Pipe security

## Access control privileges

### Creating pipes

Creating and managing pipes requires a role with a minimum of the following privileges:

| Object                             | Privilege                | Notes                 |
|------------------------------------|--------------------------|-----------------------|
| Database                           | USAGE                    |                       |
| Schema                             | USAGE,<br>CREATE<br>PIPE |                       |
| Stage in the<br>pipe<br>definition | USAGE                    | External stages only. |
| Stage in the<br>pipe<br>definition | READ                     | Internal stages only. |
| Table in the<br>pipe<br>definition | SELECT,<br>INSERT        |                       |

### Owning pipes

After a pipe is created, the pipe owner (i.e. the role that has the OWNERSHIP privilege on the pipe) must have the following privileges:

| Object                       | Privilege         | Notes                 |
|------------------------------|-------------------|-----------------------|
| Database                     | USAGE             |                       |
| Schema                       | USAGE             |                       |
| Pipe                         | OWNERSHIP         |                       |
| Stage in the pipe definition | USAGE             | External stages only. |
| Stage in the pipe definition | READ              | Internal stages only. |
| Table in the pipe definition | SELECT,<br>INSERT |                       |

## Pausing or resuming pipes

In addition to the pipe owner, a role that has the following minimum permissions can pause or resume the pipe:

| Object                       | Privilege      | Notes                 |
|------------------------------|----------------|-----------------------|
| Database                     | USAGE          |                       |
| Schema                       | USAGE          |                       |
| Pipe                         | OPERATE        |                       |
| Stage in the pipe definition | USAGE          | External stages only. |
| Stage in the pipe definition | READ           | Internal stages only. |
| Table in the pipe definition | SELECT, INSERT |                       |

## Snowpipe DDL

To support creating and managing pipes, Snowflake provides the following set of special DDL commands:

- [CREATE PIPE](#)
- [ALTER PIPE](#)
- [DROP PIPE](#)
- [DESCRIBE PIPE](#)
- [SHOW PIPES](#)

In addition, providers can view, grant, or revoke access to the necessary database objects for Snowpipe using the following standard access control DDL:

- [GRANT <privileges> ... TO ROLE](#)
- [REVOKE <privileges> ... FROM ROLE](#)
- [SHOW GRANTS](#)